Charlee44的技术驿站
聚焦于GIS、AI、计算机图形学与计算机视觉等技术的探索与实践

博客园 首页 新随笔 联系 订阅 管理 随笔 - 323 文章 - 0 评论 - 186 阅读 - 64万

3D Gaussian Splatting可视化渲染原理详解

> 本文主要围绕 **3DGS 的可视化渲染流程** 展开，希望能够帮助读者从图形学的角度理解其内部原理，而不仅仅停留在公式推导或代码实现层面。对于从事 GIS、数字孪生、三维可视化以及 AR/VR 开发的工程师而言，理解这些底层原理，也将为后续将 3DGS 与传统图形引擎融合、构建更复杂的实时三维应用打下基础。

一、引言

提到当下计算机视觉与图形学领域最炙手可热的技术，非 **3D Gaussian Splatting（简称 3DGS，3D 高斯泼溅）** 莫属。

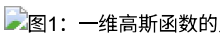
用最通俗的话来定义，3DGS 是一种利用数百万个**可学习的 3D 色块**来实时渲染逼真 3D 场景的技术。在具体的工程实践中，3DGS 的数据载体通常是一个 `.ply` 文件。这个文件里记录的，本质上是一堆带有丰富属性（如位置、颜色、透明度等）的点云数据。那么，一个极其关键的问题来了：**如何将这种常规意义上的离散点云，在屏幕上渲染成连续且逼真的三维画面**？答案就在于这个点云文件中的每个点，其实并不是一个常规意义上的物理位置点，而是一个**3D 高斯椭球体**。

二、高斯函数

要理解什么叫做 3D 高斯椭球体，就必须先明白这里的“高斯”到底指的是什么。其实并不复杂，它就是我们在数学和概率论中经常接触到的高斯函数。我们可以从一维到三维，一步步递推来看。

1. 一维高斯函数

在几何形式上，一维高斯函数表现为一条经典的**钟形曲线**，如下图所示：


图1：一维高斯函数的几何表示是一条钟形曲线

在代数表达上，一维高斯函数定义为：

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{1}$$

其中：

- μ：中心点（均值）。
- σ^2：方差。它决定了这条曲线的“胖瘦”。σ^2 越大，曲线越扁平；越小，曲线越尖锐。
- $\exp(\dots)$：就是自然常数 e 的指数次幂。

仔细观察，指数部分 $(x-\mu)^2$ 其实就是 1x1 矩阵的乘法。我们可以把它写成向量形式：

$$f(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2}(x-\mu)^T(\sigma^2)^{-1}(x-\mu)\right) \tag{2}$$

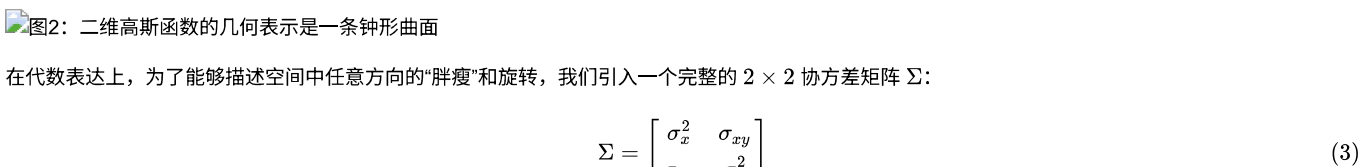
高斯函数在科学与工程领域中非常重要，使用的地方非常多。比如在统计学中，它定义了正态分布——这是自然界中最普遍、最符合直觉的概率分布。根据中心极限定理，无数独立随机变量的总和最终都会趋向于高斯分布，使其成为描述自然现象的最自然法则。再比如，在信号与图像处理中，高斯函数具备极其优异的数学性质：它不仅极其平滑，而且是最理想的低通滤波器，能够有效滤除高频噪声并带来极佳的线性平滑特性；更神奇的是，高斯函数在空间域和频域下都保持钟形，是连接物理世界与数字信号处理的最佳桥梁。

2. 二维高斯函数

(1) 定义

现在升维到二维空间，此时一个点的位置变成了向量 $\mathbf{x} = \begin{bmatrix} x \\ y \end{bmatrix}$，中心点变成 $\boldsymbol{\mu} = \begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix}$。

在几何形式上，二维高斯函数表现为一个中心凸起、向四周平滑过渡的**钟形曲面**，如下图所示：


图2：二维高斯函数的几何表示是一条钟形曲面

在代数表达上，为了能够描述空间中任意方向的“胖瘦”和旋转，我们引入一个完整的 2×2 协方差矩阵 Σ：

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} \\ \sigma_{xy} & \sigma_y^2 \end{bmatrix} \tag{3}$$

其中，对角线上的 σ_x^2 和 σ_y^2 控制着曲面在两个主轴方向上的宽度，而非对角线上的 σ_{xy}（协方差）则控制着这个高斯分布的**旋转角度**。

此时，二维高斯函数就可以被极其优雅地写成与一维完全统一的矩阵形式：

$$f(\mathbf{x}) = \frac{1}{2\pi\sqrt{\det(\Sigma)}} \exp\left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right) \tag{4}$$

这样，无论是一维还是二维，公式的外壳完全一样，只是向量 $\mathbf{x}$ 的维度变长了，标量方差 σ^2 变成了协方差矩阵 Σ。

(2) 协方差矩阵

在一维空间中，数据只沿着一条直线分布，我们只需要一个标量“方差 σ^2”就能描述它偏离中心的程度。但在二维或多维空间中，变量之间往往存在相互关联。例如，一个点不仅在 x 方向上分散，在 y 方向上分散，而且 x 和 y 的变化趋势可能还是绑定的（比如 x 变大时，y 也倾向于变大）。

为了全面描述这种多维空间中各个方向上的“胖瘦（方差）”以及维度之间的“相关性（协方差）”，一维的标量方差就不够用了，必须升级为一个矩阵——**协方差矩阵** Σ。它不仅记录了每个维度的独立方差（对角线元素），还记录了维度间的协同变化（非对角线元素），从而能够精准地刻画高斯分布在多维空间中的整体形态和旋转姿态。

协方差矩阵有一种常用的特殊情况：当 x 和 y 两个维度的变化趋势完全独立，没有任何绑定时，它们之间的协方差 $\sigma_{xy} = 0$。此时，协方差矩阵退化为了一个**对角矩阵**：

$$\Sigma = \begin{bmatrix} \sigma_x^2 & 0 \\ 0 & \sigma_y^2 \end{bmatrix} \tag{5}$$

此时，二维高斯函数就可以表达成类似于一维高斯函数式 (1) 的形式：

$$f(x, y) = \frac{1}{2\pi\sigma_x\sigma_y} \exp\left(-\frac{1}{2} \left[\frac{(x-\mu_x)^2}{\sigma_x^2} + \frac{(y-\mu_y)^2}{\sigma_y^2} \right] \right) \tag{6}$$

这种展开的表达方式在早期的教材和基础算法中非常常见，因为它不需要动用矩阵乘法，物理意义一目了然。

(3) 高斯椭圆

从图2中可以直观的看到：用于表达二维高斯函数的钟形曲面，其**等势面（即空间中高度相同的点连成的轨迹）是一个标准的椭圆**。在实际工程中的算法计算和几何控制上，这个高斯椭圆更能代表高斯函数。我们可以从代数公式上直接看出这种联系，在二维高斯函数的指数部分：

$$-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \tag{7}$$

如果我们令指数部分等于一个常数 C（即截取某一特定高度的等高线），由于 Σ^{-1} 是一个对称正定矩阵，这个方程在几何上恰好是一个**标准的二次型椭圆方程**。

这意味着，协方差矩阵 Σ 本质上决定了这个椭圆的形状与朝向：

- **椭圆的形状（长短轴比例）**：由矩阵 Σ 的特征值决定。特征值越大，说明该方向上的方差越大，椭圆在这个方向上就越“胖”（被拉伸）。
- **椭圆的朝向（旋转角度）**：由矩阵 Σ 的特征向量决定。特征向量指示了椭圆的长短轴方向。当非对角线元素 $\sigma_{xy} \neq 0$ 时，特征向量会发生倾斜，整个椭圆也就随之在平面上旋转。

而椭圆的其他性质则由公式中的其余参数确定：

- **椭圆的大小（面积）**：由指数部分的常数 C 决定。C 相当于在钟形曲面上截取等高线的高度阈值，C 越小（越靠近山顶），截出的椭圆面积越小；C 越大（越靠近山脚），椭圆面积越大。这些大小不同但形状相同的椭圆层层嵌套，共同构成了钟形曲面的等势面族。
- **椭圆的绝对位置**：由均值向量 $\boldsymbol{\mu} = \begin{bmatrix} \mu_x \\ \mu_y \end{bmatrix}$ 决定。$\boldsymbol{\mu}$ 是公式中唯一的平移项，它将整个椭圆从坐标原点“搬”到了二维空间中的指定位置，即所有同心椭圆的公共中心点。

(4) 归一化系数

无论是公式 (2) 中的 $\sqrt{2\pi}\sigma$，还是公式 (4) 中的 $2\pi\sqrt{\det(\Sigma)}$，公式最前面的分母都充当着“归一化系数”的角色。这个系数的存在，是为了保证一个铁律：**高斯函数在全空间的积分必须严格等于 1**。

如果要从数学上严格证明这一点，需要利用二重积分将直角坐标系转换为极坐标系来求解，过程相对繁琐。但在这里，我们只需要定性地理解这一点：**在多维空间中，矩阵的行列式 $\det(\Sigma)$ 本质上代表了该矩阵对“空间体积（或面积）”的绝对缩放比例**。

在二维空间中，如果协方差矩阵 Σ 的特征值分别是 λ_1 和 λ_2，这意味着空间在两个主轴方向上分别被拉伸了 $\sqrt{\lambda_1}$ 和 $\sqrt{\lambda_2}$ 倍。根据几何常识，一个平面图形在两个垂直方向上分别被拉伸后，其总面积的缩放倍数恰好是这两个倍数的乘积，即 $\sqrt{\lambda_1} \times \sqrt{\lambda_2} = \sqrt{\lambda_1 \lambda_2}$。而在矩阵理论中，矩阵的行列式刚好等于其特征值的乘积（$\det(\Sigma) = \lambda_1 \lambda_2$）。因此，$\sqrt{\det(\Sigma)}$ 就精准地刻画了二维高斯分布在空间中占据的“有效面积”。

理解了这一点，归一化系数的逻辑就水到渠成了：

- **在一维空间中**：方差 σ^2 决定了钟形曲线的“胖瘦”（即一维长度）。为了保证曲线下方的面积恒为 1，中心点的高度必须与宽度成反比。因此，一维的归一化系数是 $\frac{1}{\sqrt{2\pi}\sigma}$。
- **在二维空间中**：高斯分布的“胖瘦”由协方差矩阵 Σ 决定。当高斯椭圆在空间中变得越扁长（即 $\det(\Sigma)$ 变大，占据的面积变大）时，为了保证整个钟形曲面向下积分的总体积依然是 1，它的中心峰值高度就必须相应地降低。

因此，二维的归一化系数必须除以这个“面积因子”。结合一维公式中自带的常数 2π，二维高斯函数的归一化系数就自然而然地变成了 $\frac{1}{2\pi\sqrt{\det(\Sigma)}}$。无论高斯椭球如何被拉伸或旋转，这个系数都能极其聪明地将“各个方向的拉伸”和“空间的旋转”打包在一起，精准地算出它真实的几何体积，从而完美地维持能量守恒。

3. 三维高斯函数

(1) 定义

理解了二维高斯函数，三维高斯函数也就简单了。在三维空间中，点变成了三维向量 $\mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$，中心点变成了 $\boldsymbol{\mu} = \begin{bmatrix} \mu_x \\ \mu_y \\ \mu_z \end{bmatrix}$。

在代数表达上，为了能够描述空间中任意方向的“胖瘦”以及复杂的旋转姿态，协方差矩阵也必须升级为 3×3 的方阵：

$$\Sigma = \begin{bmatrix} \sigma_x^2 & \sigma_{xy} & \sigma_{xz} \\ \sigma_{xy} & \sigma_y^2 & \sigma_{yz} \\ \sigma_{xz} & \sigma_{yz} & \sigma_z^2 \end{bmatrix} \tag{8}$$

此时，三维高斯函数的代数表达为：

$$f(\mathbf{x}) = \frac{1}{\sqrt{(2\pi)^3 \det(\Sigma)}} \exp\left(-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right) \tag{9}$$

可以看到，公式的外壳依然没有变，只是维度再次变长了。归一化系数中的 $(2\pi)^2$ 变成了 $(2\pi)^3$，而分母中的根号依然是 $\det(\Sigma)$，因为三维空间中的体积缩放比例，同样由矩阵的行列式决定。

(2) 高斯椭球

在几何形式上，二维高斯函数是一个三维的钟形曲面，三维高斯函数的几何表达则是一个四维超曲面。这个四维超曲面超出人类三维的视觉极限，所以无法展示相应的示意图。但是我们真正用来表达它的几何形态，是其等势体——一个中心凸起、向四周平滑过渡的三维椭球体。

在二维空间中，等势面是椭圆；那么在三维空间中，如果我们令指数部分等于常数 C，由于 3×3 的 Σ^{-1} 依然是对称正定矩阵，这个方程在几何上就定义了一个**标准的三维椭球体**。与二维椭圆完全对应，这个三维椭球的几何属性依然由协方差矩阵 Σ 掌控：

- **椭球的形状（三个主轴的长度）**：由矩阵 Σ 的三个特征值决定。特征值越大，说明该方向上的方差越大，椭球在这个方向上就被拉得越长。
- **椭球的朝向（空间中的旋转姿态）**：由矩阵 Σ 的三个特征向量决定。它们指示了椭球三个主轴在三维空间中的绝对朝向。

三、高斯椭球体

1. 椭球参数

一个典型的 3DGS `.ply` 文件头包含了成百上千个顶点的定义。初看之下，它似乎只是一个普通的点云文件，但细究其属性字段，便会发现其中暗藏玄机。文件中关于形状的关键定义如下：

```
element vertex 6131954
property float x
property float y
property float z
property float nx
property float ny
property float nz
property float f_dc_0
property float f_dc_1
property float f_dc_2
property float f_rest_0
property float f_rest_1
property float f_rest_2
property float f_rest_3
property float f_rest_4
```

```
property float f_rest_5
property float f_rest_6
property float f_rest_7
property float f_rest_8
property float f_rest_9
property float f_rest_10
property float f_rest_11
property float f_rest_12
property float f_rest_13
property float f_rest_14
property float f_rest_15
property float f_rest_16
property float f_rest_17
property float f_rest_18
property float f_rest_19
property float f_rest_20
property float f_rest_21
property float f_rest_22
property float f_rest_23
property float f_rest_24
property float f_rest_25
property float f_rest_26
property float f_rest_27
property float f_rest_28
property float f_rest_29
property float f_rest_30
property float f_rest_31
property float f_rest_32
property float f_rest_33
property float f_rest_34
property float f_rest_35
property float f_rest_36
property float f_rest_37
property float f_rest_38
property float f_rest_39
property float f_rest_40
property float f_rest_41
property float f_rest_42
property float f_rest_43
property float f_rest_44
property float opacity
property float scale_0
property float scale_1
property float scale_2
property float rot_0
property float rot_1
property float rot_2
property float rot_3
```

如前所述， `.ply` 文件的每个点都是一个三维高斯椭球。但是，从文件的顶点属性来看，我们可以看到位置属性 $\boldsymbol{\mu}$ ，但是看不到前面提到的协方差矩阵 Σ 。其实，顶点属性中表达椭球的是 `scale` （缩放）和 `rot` （旋转）参数，它们可以由协方差矩阵分解得到。

协方差矩阵是一个**对称正定矩阵**。在线性代数中，这类矩阵有一个非常重要的性质——**特征分解（Eigen Decomposition）**：即对称正定矩阵 Σ 可以被分解为旋转和缩放两个操作的组合：

$$\Sigma = R\Lambda R^T \tag{10}$$

其中：

- R **是旋转矩阵（Rotation Matrix）**：由 Σ 的特征向量组成。它定义了椭球在空间中的**朝向**，即椭球的三个主轴（长、宽、高）指向哪个方向。
- Λ **是对角矩阵（Diagonal Matrix）**：由 Σ 的特征值组成（$\lambda_1, \lambda_2, \lambda_3$）。它定义了椭球沿这三个主轴方向的**尺度（大小）**，即椭球在该方向上拉伸或压缩的程度。

回到 `.ply` 文件的属性字段，文件并没有存储原始的矩阵数值，而是存储了计算机图形学中更常用的**几何参数**：

- `scale_0` , `scale_1` , `scale_2` ：
  这三个值对应了对角矩阵 Λ 的平方根（即标准差 σ）。它们决定了椭球在三个局部坐标轴上的**半轴长度**。数值越大，椭球在该方向上就越“胖”；数值越小，则越“瘦”。
- `rot_0` , `rot_1` , `rot_2` , `rot_3` ：
  这四个值构成了一个**单位四元数（Quaternion）**，它与旋转矩阵 R 是等价的数学表达。在计算机图形学中，四元数比矩阵更紧凑且能避免万向锁问题。它决定了椭球的**空间姿态**，即椭球是横着放、竖着放，还是任意角度倾斜。

总的来说，3DGS 技术通过这种参数化的方式，将复杂的概率密度函数（高斯分布）转化为计算机极易处理的“位置+旋转+缩放”数据结构。

2. 各向异性

(1) 数学意义

在计算机图形学中，几何体的形态特征通常分为两类：

- **各向同性**：物体在各个方向上的性质（如尺寸、密度）都是相同的。例如一个完美的球体，无论从哪个角度看，它的“宽度”都一样。
- **各向异性**：物体在不同方向上的性质表现出差异。例如一个椭球体，它在长轴方向上看起来很宽，但在短轴方向上看起来很窄。

在 3DGS 的语境下，**各向异性指的就是高斯椭球体在三维空间中呈现出的这种“非均匀”的拉伸或压缩特性**。这种特性并非随意产生，而是由协方差矩阵 Σ 的内部结构严格决定的。以式 (8) 的三维高斯函数的协方差矩阵为例：

1. **对角线元素（$\sigma_x^2, \sigma_y^2, \sigma_z^2$）—— 方差的大小**：
   它们决定了椭球体在 x, y, z 三个坐标轴方向上的**扩散程度**。如果这三个值相等（$\sigma_x^2 = \sigma_y^2 = \sigma_z^2$），那么该高斯点就是一个**各向同性**的球体。但在 3DGS 中，我们通常需要让椭球在某些方向上更“胖”（方差大），在某些方向上更“瘦”（方差小），从而形成椭球的**长短轴**。这种方差在不同维度上的不均衡分布，是产生各向异性的基础。
2. **非对角线元素（$\sigma_{xy}, \sigma_{xz}, \sigma_{yz}$）—— 协方差的耦合**：
   这些元素描述了不同维度之间的**相关性**。如果非对角线元素全为 0，椭球的主轴将平行于坐标轴；如果非对角线元素不为 0，意味着 x 的变化会带动 y 或 z 的变化，这种维度间的“联动”会导致椭球在空间中发生**旋转**，使其主轴指向任意空间方向。

通过特征分解式 (10)，我们将这种复杂的耦合关系解耦为直观的**旋转（R）**和**缩放（Λ）**：

$$\Sigma = R\Lambda R^T \tag{11}$$

- **缩放矩阵 Λ**：直接量化了各向异性的**强度**。Λ 的三个对角线元素（特征值）之比，就是椭球长短轴之比。如果三个特征值差异巨大，说明该高斯点具有极强的各向异性（例如一个极扁的盘状或极长的棒状）。
- **旋转矩阵 R**：决定了各向异性**的方向**。它将椭球的主轴（由 Λ 定义的最长、最短方向）旋转到世界坐标系中的正确朝向。

(2) 物理意义

在 3DGS 的渲染过程中，各向异性赋予了算法“理解几何细节”的能力：

- **边缘保持（Edge Preservation）**：在物体的边缘或角落处，光线的变化是剧烈且具有方向性的。各向异性允许高斯椭球在边缘方向上拉伸（大半轴），在垂直于边缘的方向上压缩（小半轴）。这样，一个各向异性的椭球就能完美地贴合物体的锐利边缘，而不会像各向同性的球体那样在边缘处产生模糊的“光晕”伪影。
- **表面拟合（Surface Fitting）**：对于光滑的曲面，各向异性椭球可以像“瓦片”一样平铺在表面上，其扁平的形态能更好地模拟表面的切平面，从而在较少的点数下实现极高的视觉保真度。
- **细节表达（Detail Representation）**：各向异性是 3DGS 能够从简单的“点云”进化为复杂的“连续场”的关键。它使得每一个高斯点不再是一个模糊的色块，而是一个具有明确几何指向的微小图元。

总结来说，各向异性是 3DGS 的“画笔”。通过精确控制每一个高斯椭球的 Σ 矩阵（即 `.ply` 文件中的 `scale` 和 `rot`），我们实际上是在控制这支画笔的笔触——是画一个圆点，还是画一笔长线条，或者是铺一片平面。正是这种对空间分布的精细控制，使得 3DGS 能够以极高的效率和质量重建出复杂的 3D 场景。

3. 高斯的本质

在理解了高斯椭球体的数学构造与各向异性特性后，就能理解为什么可以使用“高斯椭球”来表达可视化场景的原因了。

当我们用相机拍摄物体时，受限于镜头的光圈衍射、景深限制以及物体表面微观几何的粗糙度，光线在空间中的传播和汇聚从来都不是理想化的“锐利边界”，而是充满了平滑的过渡和不可避免的模糊。而高斯函数则可以比较好地契合这个成像规律：能量（或概率）高度集中在中心点 $\boldsymbol{\mu}$，且随着与中心点距离的增加呈指数级衰减，边缘的值迅速趋近于 0。

高斯椭球体本质上是一种基于物理的辐射场表达方式，是一团“边缘柔和的发光云”：每一个高斯点都在空间中定义了一个连续的、可微的辐射场。当无数个这样的高斯云在空间中叠加时，它们能够自然地融合与过渡，无需任何额外的抗锯齿处理，就能复现出照片级的真实感。

四、高斯投影

已经了解了三维高斯椭球体的本质，那么 3DGS 可视化是不是就是直接在 GPU 中绘制一堆三维椭球，然后进行深度排序和透明度混合呢？

理论上可行，但这在实际工程中是**无法做到**的。原因有二：一是性能瓶颈，数百万个动态图元的深度排序和混合会瞬间拖垮渲染管线；二是硬件限制，现代光栅化硬件最擅长处理的是三角形，而不是这种数学定义的曲面体。

因此，3DGS 采取了一种极其聪明的**视平面投影**策略。它的核心思想是：**既然最终画面是二维的，我们何不直接计算出每个 3D 高斯椭球在当前摄像机视角下，投影到屏幕上的 2D 形态，然后像绘制普通 2D 图像一样去绘制它？**

我们将其转换成数学模型，即一个三维高斯椭球由其在三维空间中的协方差矩阵 Σ 定义；当我们将这个椭球投影到二维图像平面时，它会变成一个二维高斯椭圆。这个新的二维椭圆，同样可以由一个 2×2 的协方差矩阵 Σ' 来描述。那么，如何从 Σ 得到 Σ' 呢？这里就涉及到了**雅可比矩阵（Jacobian Matrix）**。

1. 雅可比矩阵

想象一下，我们将三维空间中的一个点 $\mathbf{x}$ 通过一个变换函数 T 映射到二维图像平面上的点 $\mathbf{x}'$。这个变换 T 包含了从世界坐标系到相机坐标系，再到图像坐标系的整个过程（即模型-视图-投影变换）。

$$\mathbf{x}' = T(\mathbf{x}) \tag{12}$$

很显然，由于透视投影的存在，这是一个复杂的**非线性变换**。为了研究它，我们无法直接处理全局的弯曲，只能退而求其次，研究其**局部线性**的性质。类比来说，就好像在地球表面上行走时，我们感觉地面是平的。

在数学上，我们可以利用**泰勒公式**对变换函数 T 在点 $\mathbf{x}$ 附近进行一阶展开。忽略高阶无穷小量后，变换的增量 $\Delta\mathbf{x}'$ 与 $\Delta\mathbf{x}$ 之间近似满足线性关系：

$$\Delta\mathbf{x}' \approx \frac{\partial T}{\partial \mathbf{x}} \Delta\mathbf{x} \tag{13}$$

这里的偏导数矩阵 $\frac{\partial T}{\partial \mathbf{x}}$，就是**雅可比矩阵** J。它本质上是变换函数 T 在局部的一阶导数：

$$J = \frac{\partial T}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial x'}{\partial x} & \frac{\partial x'}{\partial y} & \frac{\partial x'}{\partial z} \\ \frac{\partial y'}{\partial x} & \frac{\partial y'}{\partial y} & \frac{\partial y'}{\partial z} \end{bmatrix} \tag{14}$$

这个 2×3 的矩阵 J 告诉我们，三维空间中的一个微小变化（或扰动），会如何线性地映射并影响二维图像平面上的位置。正是通过这种“局部线性化”的手段，我们将复杂的非线性投影问题，转化为了简单的矩阵乘法问题。

2. 协方差传播

在概率论和统计学中，有一个极其重要的性质，称为**协方差传播定律（Law of Propagation of Covariance）**。这个定律的完整表述是：

如果一个随机向量 $\mathbf{x}$ 的协方差矩阵是 $\Sigma_{\mathbf{x}}$，它经过一个（可微的）变换 T 得到一个新的随机向量 $\mathbf{y} = T(\mathbf{x})$，那么新向量 $\mathbf{y}$ 的协方差矩阵 $\Sigma_{\mathbf{y}}$ 可以通过以下公式近似计算：

$$\Sigma_{\mathbf{y}} \approx J \Sigma_{\mathbf{x}} J^T \tag{15}$$

其中，J 就是变换 T 在 $\mathbf{x}$ 的均值点处的**雅可比矩阵**。这里的近似条件取决于以下两种情况：

- **线性变换**：如果变换 T 是严格的线性变换（例如旋转、缩放），那么上述公式是**精确成立**的。
- **非线性变换**：如果变换 T 是非线性的（例如我们这里的透视投影），那么这个公式是一个**一阶近似**。它相当于在局部用一个切平面（由雅可比矩阵定义）来近似这个非线性的曲面。

3. 投影公式

具体到我们的场景中，三维高斯椭球的协方差矩阵 Σ 在经过投影变换 T 后，其在二维图像平面上形成的椭圆的协方差矩阵 Σ' 可以通过以下公式计算：

$$\Sigma' = J \Sigma J^T \tag{16}$$

其中，J 就是我们在上一节中提到的、投影变换在当前高斯中心点处的雅可比矩阵。

由于透视投影本质上是非线性的，上述公式其实是协方差传播定律在**一阶近似**下的应用。它相当于在局部用一个切平面（由雅可比矩阵定义）来近似这个非线性的曲面。

但在 3DGS 的实际应用中，由于每个高斯椭球本身的尺寸相对于整个宏大的场景来说非常微小，这种一阶近似已经足够精确，并且计算效率极高。

这个公式就是 3DGS 渲染管线的核心。它意味着：

1. **输入**：每个高斯椭球在三维空间中的协方差矩阵 Σ（由 `.ply` 文件中的 `scale` 和 `rot` 参数实时计算得出）。
2. **变换**：根据当前摄像机的位置和角度，计算出从世界空间到当前帧图像空间的雅可比矩阵 J。
3. **输出**：通过一次简单的矩阵乘法，得到该椭球在当前视角下应该呈现的**二维高斯椭圆**的协方差矩阵 Σ'。

通过这种方式，3DGS 极其聪明地将三维空间中的“不确定性椭球”，精准地“压扁”成了二维图像上的“不确定性椭圆”，完美地衔接了 3D 场景与 2D 屏幕。

4. 具体计算

理论虽然完美，但在代码实现中，我们需要把这个过程拆解为具体的数学步骤。整个投影过程可以概括为三步：**视图变换**、**投影变换**、**协方差传播**。

(1) 视图变换

首先，我们需要将高斯椭球从**世界坐标系**转换到**相机坐标系**。这一步非常简单，它是一个纯粹的刚体变换（旋转 + 平移），由视图矩阵 W 决定。

假设高斯椭球在世界坐标系下的中心为 $\boldsymbol{\mu}_{world}$，协方差为 Σ_{world}。经过视图变换后，它在相机坐标系下的新中心 $\boldsymbol{\mu}_{cam}$ 和新协方差 Σ_{cam} 为：

$$\boldsymbol{\mu}_{cam} = W \boldsymbol{\mu}_{world} \tag{17}$$

$$\Sigma_{cam} = W \Sigma_{world} W^T \tag{18}$$

因为视图变换是线性的，所以协方差矩阵直接左乘右乘视图矩阵即可，不需要用到雅可比矩阵。此时，我们得到了一个位于相机前方的 3D 高斯椭球。

(2) 投影变换

接下来是最关键的一步：将相机坐标系下的 3D 点 (x, y, z) 投影到 2D 图像平面 (u, v) 上。

在针孔相机模型中，这个投影过程是非线性的，其数学公式为：

$$u = f_x \frac{x}{z} + c_x \tag{19}$$

$$v = f_y \frac{y}{z} + c_y \tag{20}$$

其中：

- f_x, f_y 是相机的焦距（像素单位）。
- c_x, c_y 是图像的主点（通常是图像中心）。
- z 是深度值。

为了应用**协方差传播定律**，我们需要计算这个非线性变换在点 (x, y, z) 处的**雅可比矩阵** J。根据偏导数的定义，我们对 u 和 v 分别关于 x, y, z 求导：

- $\frac{\partial u}{\partial x} = \frac{f_x}{z}$，$\frac{\partial u}{\partial y} = 0$，$\frac{\partial u}{\partial z} = -\frac{f_x x}{z^2}$
- $\frac{\partial v}{\partial x} = 0$，$\frac{\partial v}{\partial y} = \frac{f_y}{z}$，$\frac{\partial v}{\partial z} = -\frac{f_y y}{z^2}$

将这些偏导数组合起来，我们就得到了具体的 2×3 雅可比矩阵：

$$J = \begin{bmatrix} \frac{f_x}{z} & 0 & -\frac{f_x x}{z^2} \\ 0 & \frac{f_y}{z} & -\frac{f_y y}{z^2} \end{bmatrix} \tag{21}$$

现在，我们将相机坐标系下的协方差 Σ_{cam} 和刚刚算出的雅可比矩阵 J 代入协方差传播公式：

$$\Sigma_{2D} = J \Sigma_{cam} J^T \tag{22}$$

计算出的 Σ_{2D} 就是一个 2×2 的矩阵。它描述了该高斯椭球在**图像平面**上的形状。

(3) 完整流程

为了更清晰地理解，我们可以把整个过程串联起来：

1. **输入**：从 `.ply` 文件读取 3D 高斯的参数（位置、缩放、旋转、颜色、透明度）。
2. **视图变换**：利用视图矩阵 W，将 3D 高斯从**世界空间**变换到**相机空间**，得到 Σ_{cam}。
3. **计算雅可比**：根据当前高斯在相机空间的位置 (x, y, z) 和相机内参 (f_x, f_y)，计算雅可比矩阵 J。
4. **投影**：利用公式 $\Sigma_{2D} = J \Sigma_{cam} J^T$，计算出 2D 协方差矩阵。
5. **光栅化准备**：对 Σ_{2D} 进行特征分解，得到 2D 椭圆的**长短轴**和**旋转角度**。
6. **绘制**：将这个 2D 椭圆（实际上是一个带模糊边缘的贴片）绘制到屏幕的对应像素上，并根据深度 z 进行排序和混合。

通过这一套严密的数学推导，3DGS 成功地将复杂的 3D 渲染问题，转化为了高效的 2D 图像处理问题。

五、图形实现

经过上一章的推导，我们已经知道：每一个三维高斯椭球，都能够通过雅可比矩阵投影成屏幕上的二维高斯椭圆。那么，GPU 到底应该如何把这个二维高斯椭圆绘制出来？很显然，由于我们已经通过高斯投影知道了屏幕空间的椭圆，那么就不太适合“顶点(Vertex)->图元装配->光栅化(Rasterization)->片元(Fragment)->像素(Pixel)”这一套经典的光栅化流水线过程了。

> 在论文原版 3DGS 渲染实现中，是使用 GPU Compute Shader直接写 Image 的方法来实现的。这里笔者讨论是如何利用现有的图形渲染技术进行绘制的问题，因为我们希望 3DGS 能存在于已有的图形渲染队列中。

1. 程序化渲染

如果接触过粒子系统（Particle System），会发现这里其实非常类似一种经典技术——**Billboard（广告牌）**。Billboard 的思想非常简单：GPU 并不会真正绘制一个三维物体。它只是生成一个始终朝向摄像机的二维矩形（Quad），然后在这个矩形内部完成纹理采样。例如一个火焰粒子：

```
Camera
```

```
        ↑

   +--------------+
   |              |
   |   Texture    |
   |              |
   +--------------+
```

无论摄像机如何旋转，这个 Quad 都会始终面向屏幕，因此用户看到的是一个始终正对自己的粒子。3DGS 的高斯椭圆与它非常相似，区别在于 Billboard 绘制的是一个矩形 + 一张纹理；而 Gaussian Splatting 绘制的是：一个矩形 + 一个二维高斯函数。也就是说，高斯椭圆并不是一张预先准备好的图片，而是在 Shader 中实时计算出来的连续函数。

由于二维高斯可以直接由数学公式计算，因此实际上并不存在真正意义上的"高斯模型"。GPU 真正绘制的，仍然只是一个十分普通的矩形。整个流程可以理解为：

```
Gaussian
        |
        ▼
计算二维椭圆
        |
        ▼
生成包围矩形(Quad)
        |
        ▼
Rasterization
        |
        ▼
Fragment Shader
        |
        ▼
计算二维高斯函数
        |
        ▼
输出颜色与透明度
```

真正的高斯形状，并不是由几何决定，而是由 Fragment Shader 中的数学计算决定。因此，这种方式通常也称为**程序化渲染（Procedural Rendering）**。这种技术在现代 GPU 渲染中十分常见，总结起来就是：几何尽可能简单，而图形细节由 Shader 动态生成。

2. 着色实现

在明确了“程序化渲染”的策略后，我们需要将之前推导的数学公式落地到具体的 Shader 代码中。这里笔者将整个渲染流程拆分为两部分：**CPU 端**负责投影与几何计算，**GPU 端**则专注于的图元定位与颜色混合。

(1) CPU 端

在每一帧渲染开始前，在 CPU 端遍历所有的高斯椭球，完成从 3D 到 2D 的投影计算。这一步的核心是利用雅可比矩阵计算出屏幕空间的协方差矩阵，并对其进行特征分解，从而得到绘制所需的精确几何参数。

```
// CPU 端逻辑：为每个高斯计算投影参数
for (每个高斯 gaussian) {
    // ... 省略视图变换与雅可比矩阵计算，假设已得到 2D 协方差矩阵 covariance_2d ...

    // 1. 特征分解：获取旋转和缩放
    // eigenvalues: 两个特征值 (lambda1, lambda2)
    // eigenvectors: 对应的特征向量 (组成旋转矩阵)
    auto [eigenvalues, eigenvectors] = decompose(covariance_2d);

    // 2. 计算半轴长度 (通常截取 3 倍标准差，覆盖 99% 的能量)
    vec2 semi_axes = 3.0 * sqrt(eigenvalues);

    // 3. 计算旋转角度 (从特征向量中提取)
    float rotation = atan2(eigenvectors[0].y, eigenvectors[0].x);

    // 4. 打包数据发给 GPU
    GpuData data;
    data.center = project_to_screen(position_cam); // 屏幕中心点 (u, v)
    data.semi_axes = semi_axes;       // 精确的半轴长度
    data.rotation = rotation;         // 精确的旋转角度
    data.inv_covariance = inverse(covariance_2d); // 协方差矩阵的逆，用于片元计算
    data.color = gaussian.color;
    data.opacity = gaussian.opacity; // 学习到的基础不透明度

    gpu_buffer.push(data);
}
```

(2) 顶点着色器

顶点着色器的任务非常纯粹：接收 CPU 传来的几何参数，利用 `gl_VertexID` 动态生成一个包围该高斯椭圆的矩形（Quad），并将其变换到正确的屏幕位置。

```
// 顶点着色器 (Vertex Shader)
in vec2 u_center;       // 屏幕中心
in vec2 u_semi_axes;    // 精确的半轴长度 (来自特征分解)
in float u_rotation;    // 精确的旋转角度

out vec2 v_position;    // 传递给片元着色器的局部偏移量

void main() {
    // 1. 基础矩形的四个角 (-1 到 1)
    vec2 offsets[4] = vec2[](vec2(-1,-1), vec2(1,-1), vec2(-1,1), vec2(1,1));
    vec2 local_pos = offsets[gl_VertexID];

    // 2. 缩放：将 -1~1 的范围拉伸到实际的半轴长度
    vec2 scaled_pos = local_pos * u_semi_axes;

    // 3. 旋转：应用旋转矩阵
    float cos_r = cos(u_rotation);
    float sin_r = sin(u_rotation);
    mat2 rot_matrix = mat2(cos_r, -sin_r, sin_r, cos_r);
    vec2 rotated_pos = rot_matrix * scaled_pos;

    // 4. 平移到屏幕中心
    vec2 screen_pos = u_center + rotated_pos;

    // 5. 转换为标准化设备坐标 (NDC)
    vec2 ndc = (screen_pos / u_screen_size) * 2.0 - 1.0;
    gl_Position = vec4(ndc, 0.0, 1.0);

    // 6. 传递局部偏移量 (用于片元着色器计算高斯衰减)
    v_position = rotated_pos;
}
```

(3) 片元着色器

片元着色器是真正“绘制”高斯形状的地方。对于包围盒内的每一个像素，我们需要计算它相对于椭圆中心的马氏距离（Mahalanobis Distance），并据此输出颜色和透明度。

这里有一个非常关键的细节：**我们不需要使用概率密度函数中的归一化系数** $\frac{1}{2\pi\sqrt{\det(\Sigma)}}$。在 3DGS 中，我们的目标是 Alpha 混合（Alpha Blending）而非概率估计。高斯椭圆的“浓淡”是由我们单独学习的一个参数——**不透明度（Opacity, α）** 来控制的。如果强行加入归一化系数，反而会导致颜色变得极暗，因为协方差矩阵的行列式通常很小，会把颜色“压”没。

```
// 片元着色器 (Fragment Shader)
in vec2 v_position; // 相对于中心的实际偏移量

uniform mat2 u_inv_covariance; // 协方差矩阵的逆
uniform vec3 u_color;
uniform float u_opacity; // 基础不透明度

out vec4 frag_color;

void main() {
    // 1. 计算马氏距离的平方
    // 公式: d² = (x - μ)ᵀ * Σ⁻¹ * (x - μ)
    // v_position 就是 (x - μ)
    float d_squared = dot(v_position, u_inv_covariance * v_position);

    // 2. 计算 Alpha
    // 注意：这里没有归一化系数，直接由基础不透明度控制
    float alpha = u_opacity * exp(-0.5 * d_squared);

    // 3. 裁剪：如果透明度过低，直接丢弃，提升性能
    if (alpha < 0.004) { // 0.004 约等于 1/255
        discard;
    }

    // 4. 输出最终颜色
    frag_color = vec4(u_color, alpha);
}
```

3. 补充说明

通过这套着色器组合，GPU 就能高效地将数百万个数学定义的高斯椭圆渲染到屏幕上。顶点着色器负责“画框”，片元着色器负责“填色”，最终呈现出平滑、连续的 3D 场景。

不过需要指出的是，上述代码主要侧重于原理的直观展示，属于一种**理论化实现**，旨在帮助读者理解 3DGS 的渲染逻辑。在实际的工业级工程应用中，为了追求极致的性能，实现方式可能有所不同；但其底层的数学原理——即**利用二维高斯函数进行屏幕空间投影与混合**——是完全一致的。

六、球谐系数

在上一节我们详细探讨了 3DGS 中每个高斯椭球的几何属性（位置、缩放、旋转），但一个逼真的 3D 场景显然不能只有形状，还必须有丰富的颜色和光照信息。回到第三章提到的 `.ply` 文件属性列表，你会发现除了 `scale` 和 `rot` 之外，还有一长串以 `f_dc` 和 `f_rest` 开头的属性：

```
property float f_dc_0
property float f_dc_1
property float f_dc_2
property float f_rest_0
...
property float f_rest_44
```

这些属性并非简单的 RGB 颜色值，而是**球谐函数（Spherical Harmonics, SH）系数**。在 3DGS 中，每个高斯椭球的颜色不是固定的，而是会随着观察角度的变化而变化，以此来模拟真实世界中光照与材质相互作用产生的复杂视觉效果（如镜面高光、菲涅尔效应等）。球谐函数正是实现这一“视角相关颜色”的核心数学工具。

1. 傅里叶变换

要解释什么是球谐函数，我们先了解一下类似的傅里叶变换。在数字信号处理中，傅里叶变换的核心思想是：**任何复杂的波形，都可以被拆解成无数个频率、振幅各不相同的纯正弦波的叠加。**

为了理解这个过程，我们需要先认识两个核心概念：

- **基函数 (Basis Functions)**：这些“纯正弦波”就是傅里叶变换的基函数。它们是构成复杂信号的“标准波纹”。
- **分解与重构**：傅里叶变换的过程，就是用这些标准波纹去“测量”原始信号，计算出每个频率的波纹在信号中占多少“份量”（即系数）。

在数学上，这一过程可以用以下公式严谨地表达：

$$f(x) = \sum_{n=0}^{\infty} c_n \cdot \phi_n(x) \tag{23}$$

其中：

- $f(x)$ 是我们要分析的原始复杂信号。
- $\phi_n(x)$ 就是第 n 个**基函数**（即我们提到的“标准正弦波纹”）。
- c_n 是对应的**系数**，代表了该基函数在原始信号中的权重或“份量”。

简单来说，傅里叶变换就是在一条直线上，用不同频率的正弦波作为“尺子”，去度量和表达一个复杂信号。

2. 球谐函数

现在，我们将傅里叶变换的思想从一维的“线”推广到三维空间中的“面”——一个单位球面。

在计算机图形学中，我们常常需要描述一个点周围的光照环境。光线可以从四面八方射来，这个光照信息是定义在一个球面上的函数，称为**光照函数 (Lighting Function)**。那么，如何表示这个复杂的球面函数呢？答案就是**球谐函数 (Spherical Harmonics, SH)**。

如果说傅里叶变换是用“平面正弦波”去拟合一条线上的信号，那么球谐变换就是用“**球面波纹**”去拟合一个球面上的信号。

- **基函数**：球谐函数的基函数不再是正弦波，而是一系列定义在球面上的、具有不同频率和方向的“波纹图案”。
- **0 阶 SH**：像一个纯色的球，代表整个球面的平均光照（环境光）。
- **1 阶 SH**：像一个从黑到白渐变的球，能表示一个主方向的光照（比如太阳从上方照射）。
- **2 阶及更高**：图案越来越复杂，能表示更精细的光照细节和反射。

因此，球谐函数本质上就是“**球面上的傅里叶变换**”。它用一组预定义的球面基函数，将复杂的环境光照压缩成一小撮系数。

在数学上，球谐函数的展开形式与傅里叶变换高度相似，其公式为：

$$f(\theta, \phi) \approx \sum_{l=0}^{n} \sum_{m=-l}^{l} c_l^m Y_l^m(\theta, \phi) \tag{24}$$

其中：

- $f(\theta, \phi)$ 是定义在球面上的原始函数（如光照分布）。

- $Y_l^m(\theta, \phi)$ 是**球谐基函数**，由阶数 l 和频率 m 共同决定。
- c_l^m 是对应的**球谐系数**，代表了该基函数在原始球面函数中的权重。

进一步地，球谐基函数 $Y_l^m(\theta, \phi)$ 的通用数学定义由三个部分组成：

$$Y_l^m(\theta, \phi) = K_l^m \cdot P_l^m(\cos\theta) \cdot e^{im\phi} \tag{25}$$

- K_l^m **（归一化常数）**：这是一个为了保证数学严谨性而存在的系数，确保基函数在球面上的积分为 1。具体计算式为：$K_l^m = \sqrt{\frac{2l+1}{4\pi}\frac{(l-m)!}{(l+m)!}}$
- $P_l^m(\cos\theta)$ **（关联勒让德多项式）**：这是决定球谐函数“形状”的核心部分。不同的阶数 l 和频率 m 会对应不同的多项式形状（如常数、线性、二次曲面等）。
- $e^{im\phi}$ **（复指数项）**：这一项代表了函数在水平方向（方位角 ϕ）上的旋转变化。

在渲染时，我们只需根据观察方向，用这些系数和基函数进行加权求和，就能快速还原出该方向上的光照强度。

3. PBR中的球谐函数

在计算机图形学的基于物理的渲染（PBR）中，球谐函数扮演着至关重要的角色，主要用于高效地计算**间接漫反射光照**。

当一个物体（如角色、车辆）在场景中移动时，它不仅要被太阳光（直接光）照亮，还会被周围环境（如墙壁、地面）反射的光线照亮。这部分光线是柔和且无明确方向的漫反射光。为了实现这个效果，图形引擎会在场景中预先放置一些**光照探针（Light Probes）**。这些探针会“捕捉”周围环境的光照信息，并将其编码为一组球谐系数。当动态物体经过时，GPU 会根据其位置插值出对应的 SH 系数，并实时计算出物体表面接收到的环境光。

这样做的好处是，用极少的计算量（通常只需 2 阶 9 个系数）就能模拟出非常逼真的全局光照效果，让动态物体完美地融入静态场景。

4. 3DGS中的球谐函数

在 3DGS 中，球谐函数的用途与 PBR 略有不同。它不是用来计算环境光，而是直接用来表示**每个高斯椭球自身的视角相关颜色（View-dependent Color）**。

3DGS 的核心思想是，一个高斯点在不同视角下看起来颜色是不同的，这模拟了现实世界中物体表面的**高光（Specular Highlight）**和**菲涅尔效应（Fresnel Effect）**。例如，一个光滑的桌面，从某个角度看会有刺眼的反光，换个角度则没有。

因此，在 3DGS 训练阶段，算法会为每个高斯点学习一组球谐系数（存储在 `.ply` 文件的 `f_rest` 字段中）。这组系数编码了该点在所有方向上的颜色变化规律。而在渲染阶段，当摄像机观察这个高斯点时，GPU 会根据**观察方向（View Direction）**和这组 SH 系数，实时计算出该点在当前视角下应该呈现的颜色。

为了平衡渲染质量与性能，3DGS 通常默认使用 3 阶球谐函数。这意味着公式 (24) 中的求和上限 n 取值为 3，阶数 l 的取值范围是 $0, 1, 2, 3$。根据球谐函数的性质，对于每一个阶数 l，频率 m 的取值范围是 $-l$ 到 l。因此，总的基函数数量（即系数个数）为 1 + 3 + 5 + 7 = 16 个。

将这 16 个基函数项代入公式 (24)，我们可以将其展开为四个部分的加权和：

$$f(\theta, \phi) \approx \underbrace{\sum_{m=0}^{0} c_0^m Y_0^m}_{\text{0 阶: 基础漫反射}} + \underbrace{\sum_{m=-1}^{1} c_1^m Y_1^m}_{\text{1 阶: 线性方向}} + \underbrace{\sum_{m=-2}^{2} c_2^m Y_2^m}_{\text{2 阶: 二次曲面}} + \underbrace{\sum_{m=-3}^{3} c_3^m Y_3^m}_{\text{3 阶: 三次细节}} \tag{26}$$

具体展开后的 16 项如下：

$$\begin{aligned} f(\mathbf{v}) \approx \quad & c_0^0(0.28) \\ & + c_1^{-1}(-0.48y) + c_1^0(0.48z) + c_1^1(-0.48x) \\ & + c_2^{-2}(1.09xy) + c_2^{-1}(-1.09yz) + c_2^0(0.31(3z^2-1)) \\ & + c_2^1(-1.09xz) + c_2^2(0.54(x^2-y^2)) \\ & + c_3^{-3}(-0.59(3x^2-y^2)y) + c_3^{-2}(2.89xyz) \\ & + c_3^{-1}(-0.45y(4z^2-x^2-y^2)) + c_3^0(0.39z(5z^2-3)) \\ & + c_3^1(-0.45x(4z^2-x^2-y^2)) + c_3^2(1.44z(x^2-y^2)) + c_3^3(-0.59x(x^2-3y^2)) \end{aligned} \tag{27}$$

在这个公式中：

- **常数项**：这些数字并非随意指定，而是根据球谐基函数的通用数学定义，代入不同的阶数 l 和频率 m 后，计算出的**归一化系数**。它们确保了基函数在球面上的能量分布是标准的。
- x, y, z **多项式**：球谐基函数原本是基于球体极坐标系（θ, ϕ）定义的，包含复杂的三角函数（如 $\sin\theta\cos\phi$）。但在 3D 图形学的实际应用中，为了方便 GPU 进行高效的矩阵运算，我们通常会将这些极坐标下的三角函数，通过数学变换“翻译”成三维笛卡尔坐标系下的多项式形式。
- **观察方向**：球谐函数本身是定义在“方向”上的，3DGS 中计算的是“当前视角”下的颜色，所以这个“方向”就是摄像机的观察方向，也就是这里的笛卡尔坐标 (x, y, z)。

- **RGB 三通道**：上述公式中的每一个系数 c_l^m 实际上都是一个包含 R、G、B 三个分量的向量。因此，对于 RGB 三个通道，总共需要存储 $16 \times 3 = 48$ 个浮点数。

这使得 3DGS 能够用极低的存储开销（使用 3 阶 SH，共 16 个系数，RGB 三通道共 48 个 float 值），就模拟出复杂的光照交互效果，让重建的场景看起来富有光泽和质感，而不是像塑料一样平淡。

5. 3DGS 着色伪代码

结合前面的知识，我们可以写出 3DGS 中计算最终颜色的核心伪代码。这个过程发生在片元着色器（Fragment Shader）中：

```
// 片元着色器伪代码

// 输入
vec3 view_direction;      // 观察方向
float sh_coeffs[48];      // 完整的球谐系数 (f_dc_0~2 + f_rest_0~44)
float opacity;            // 基础不透明度

// 1. 计算 16 个球谐基函数值 (基于观察方向)
float sh_basis[16];
sh_basis[0]  = 0.282095; // 0 阶 (对应 f_dc)
sh_basis[1]  = -0.488603 * view_direction.y; // 1 阶 (对应 f_rest 前3个)
sh_basis[2]  = 0.488603 * view_direction.z;
sh_basis[3]  = -0.488603 * view_direction.x;
// ... 计算 2 阶和 3 阶的基函数 (对应 f_rest 后42个)

// 2. 统一计算最终颜色
// 将 48 个系数与 16 个基函数进行加权求和
vec3 final_color = vec3(0.0);
for (int i = 0; i < 16; i++) {
    // RGB 三个通道分别计算
    final_color.r += sh_coeffs[i] * sh_basis[i];
    final_color.g += sh_coeffs[i + 16] * sh_basis[i];
    final_color.b += sh_coeffs[i + 32] * sh_basis[i];
}

// 3. 计算最终 Alpha
float alpha = opacity * exp(-0.5 * d_squared);

// 输出
frag_color = vec4(final_color, alpha);
```

通过这套机制，3DGS 巧妙地将复杂的、视角相关的光照效果“烘焙”进了一小组球谐系数中，实现了在极低计算成本下的高质量实时渲染。

七、透明混合

前面已经介绍了每一个高斯椭球如何计算颜色与透明度，但真正决定最终画面效果的，并不是某一个高斯自身，而是**数百万个高斯之间如何进行颜色混合（Blending）**。

事实上，这个问题并不是 3DGS 独有的问题，而是整个计算机图形学几十年来一直存在的问题。现代 GPU 的透明渲染、粒子系统、体渲染（Volume Rendering），乃至 3DGS，本质上都遵循着同一套透明度混合原理。

1. 图形渲染中的透明混合

现代 GPU 的渲染流程通常分为两个阶段：首先绘制所有**不透明物体（Opaque）**，利用深度缓冲（Depth Buffer）完成遮挡关系；随后再绘制所有**透明物体（Transparent）**。

这样做的原因很简单，对于不透明物体而言，一个像素最终只能保留距离摄像机最近的颜色，因此 GPU 可以利用 **Z-Buffer** 快速完成遮挡测试：

```
Camera

A
|
|
B

最终看到 A
```

而透明物体则完全不同。假设有两块透明玻璃：

```
Camera

Glass A   α=0.5
|
```

```
|
Glass B   α=0.5
```

此时最终颜色应当同时受到两块玻璃影响，而不是只保留最近的一块。

因此，透明物体不能简单依赖深度测试，而必须进行**透明度混合（Alpha Blending）**。

2. Alpha Blending

现代 GPU 默认采用的是经典的 **Source Over** 混合公式。设：

- 当前绘制颜色（Source）为 C_s，透明度为α_s。
- 帧缓冲已有颜色（Destination）为 C_d, 透明度为α_d。

那么混合后的颜色为：

$$C_{out} = \alpha_s C_s + (1 - \alpha_s) C_d \tag{28}$$

对应透明度为：

$$\alpha_{out} = \alpha_s + (1 - \alpha_s) \alpha_d \tag{29}$$

GPU 中对应的 OpenGL 设置通常就是：

```
glEnable(GL_BLEND);

glBlendFunc(
    GL_SRC_ALPHA,
    GL_ONE_MINUS_SRC_ALPHA
);
```

Unity、Unreal、Vulkan、DirectX 等现代图形 API 本质上都是这一套公式。

从上述公式可以看出，Alpha Blending 是**不满足交换律**的，两个透明的物体谁前谁后对最终混合的颜色影响巨大。因此透明物体的渲染队列必须遵循由远到近的顺序，这就说经典的**画家算法（Painter's Algorithm）**。它的思想非常符合现实：像画画一样，先画远处，再画近处。

3. 3DGS中的透明混合

3DGS 的透明混合与传统 GPU 几乎完全一致。区别仅在于传统图形学混合的是**三角形片元（Fragment）**；而 3DGS 混合的是**二维高斯椭圆（Gaussian Splat）**。

设第 (i) 个高斯在当前像素上的透明度为 α_i，颜色为 c_i，若所有高斯已经按照深度完成排序，则最终颜色可写成：

$$C = \sum_{i=1}^{N} T_i \alpha_i c_i \tag{30}$$

其中:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \tag{31}$$

表示第 (i) 个高斯之前剩余的透射率（Transmittance）。这个公式与传统 Alpha Blending 完全一致，只不过将 GPU 固定功能中的混合操作显式写成了数学递推。具体的含义可以这样理解：**最终颜色 = 每一个高斯球的颜色贡献 × 它前面所有高斯球的累积透明度，然后把所有高斯球的贡献加起来**。

4. 图形渲染实现

从数学角度来看，3DGS 并没有发明一种新的透明混合算法。与传统图形渲染最大的区别在于**透明度的来源不同**。在 3DGS 中，每一个二维高斯都会根据高斯函数实时计算当前像素的透明度：

$$\alpha = \alpha_0 \exp\left(-\frac{1}{2} d^2\right) \tag{32}$$

其中 α_0 为训练得到的不透明度参数，(d) 为当前像素到高斯中心的马氏距离。因此，高斯自身天然就是一个连续变化的半透明体，而无需依赖额外的 Alpha 纹理。

从工程实现来看，这种做法也带来了新的挑战。传统渲染管线依赖 GPU 固定功能完成透明混合，而 3DGS 通常需要自行控制高斯排序、颜色累积以及透射率计算，因此很难直接融入已有的 Forward 或 Deferred 渲染流程。

对于需要与传统三角形场景共同渲染的应用（如 AR、数字孪生、游戏引擎等），一种较为合理的方案是在现有渲染管线中增加**连续后处理（Post Process）**或**自定义渲染队列（Custom Render Queue）**，将 3DGS 作为独立的透明渲染阶段，与传统光栅化渲染协同工作，而不是完全替代现有的图形流水线。

5. 实现伪代码

在常规 GPU 图形渲染中，我们无法像 CPU 那样简单地用一个变量去循环累加颜色。因此，3DGS 需要借助额外的纹理来在像素之间传递混合状态。

通常，我们会引入两张关键的纹理作为额外的渲染目标：

- **累积颜色纹理（Accumulated Color Texture）**：存储公式 (30) 中的最终颜色 C。
- **累积透射率纹理（Accumulated Transmittance Texture）**：存储公式 (31) 中的剩余透射率 T（初始值为 1.0）。

以下是 3DGS 渲染单个高斯椭球时的片元着色器（Fragment Shader）核心伪代码：

```
// 片元着色器伪代码

// 输入
uniform sampler2D accumulated_color_tex; // 累积颜色纹理
uniform sampler2D accumulated_transmittance_tex; // 累积透射率纹理
in vec2 screen_uv; // 当前像素的屏幕坐标
in vec3 gaussian_color; // 当前高斯的颜色 c_i
in float gaussian_alpha; // 当前高斯在当前像素的透明度 α_i

void main() {
    // 1. 读取当前像素之前的累积状态
    vec3 C_accum = texture(accumulated_color_tex, screen_uv).rgb;
    float T_accum = texture(accumulated_transmittance_tex, screen_uv).r;

    // 2. 计算当前高斯的贡献
    // 当前高斯的颜色贡献 = 自身颜色 * 自身透明度 * 之前所有高斯的累积透射率
    vec3 contribution = gaussian_color * gaussian_alpha * T_accum;

    // 3. 更新累积颜色
    vec3 C_new = C_accum + contribution;

    // 4. 更新累积透射率
    // 新的透射率 = 旧的透射率 * (1 - 当前高斯的透明度)
    float T_new = T_accum * (1.0 - gaussian_alpha);

    // 5. 将新的状态写回纹理，供下一个高斯读取
    // 注意：实际工程中是写出到绑定的渲染目标
    imageStore(accumulated_color_image, pixel_coord, vec4(C_new, 1.0));
    imageStore(accumulated_transmittance_image, pixel_coord, T_new);
}
```

在这段伪代码实现中，关键在于以下两点：

1. **计算贡献**：根据公式 (30) 和 (31)，当前高斯对最终颜色的贡献是 `gaussian_color * gaussian_alpha * T_accum`。这里的 `T_accum` 完美对应了公式中的 $\prod_{j=1}^{i-1}(1-\alpha_j)$。
2. **累加与更新**：将贡献值加到 C_{accum} 上，并将 T_{accum} 乘以 $(1-\alpha_i)$ 得到新的透射率。

八、深度排序

上一节介绍了过 3DGS 的透明混合公式成立的前提是：**所有 Gaussian 必须按照距离摄像机由远到近（Back-To-Front）完成排序**。否则由于 Alpha Blending 不满足交换律，会出现透明颜色错误、遮挡关系异常等问题。

1. 排序依据

一个 Gaussian 本身是一个三维椭球，而不是一个点，那么它究竟按照什么深度排序？原版的 3DGS 没有考虑那么复杂，**仅使用 Gaussian 中心在相机空间中的深度作为排序依据**。

设高斯中心为 $\mu=(x,y,z)$, 经过视图变换后得到相机空间坐标：$\mu_c=(x_c,y_c,z_c)$ 。排序使用的深度就是：

$$Depth=z_c \tag{33}$$

按照 Depth 从远到近（Back-To-Front）排序。

伪代码如下：

```
for (Gaussian g)
{
    vec3 centerView =
        ViewMatrix * g.mean;

    g.depth = centerView.z;
}
```

```
Sort(depth);
```

也就是说，一个 Gaussian 是否排在前面，与它的协方差矩阵、椭球长短轴以及屏幕覆盖面积均无关系，而仅取决于它中心点距离摄像机的远近。从理论上来说，仅使用中心点排序确实不是完全准确，但这种误差通常不会十分明显，不会影响最终渲染效果，是精度与效率之间的一种工程折中。

2. 全局排序

明确了排序依据，那么是不是对所有 Gaussian 按照深度排序，再依次进行透明混合就可以了呢？我们知道，一般比较好的排序算法复杂度是 $O(N \log N)$，这对于百万级甚至千万级 Gaussian 来说，开销十分巨大。

更重要的是，由于摄像机可以自由移动，每一帧所有 Gaussian 到摄像机的距离都会发生变化，因此排序结果也必须实时更新。如果采用传统 CPU 排序：

```
Camera Move
        ↓
CPU Sort
        ↓
Upload GPU
        ↓
Render
```

不仅排序耗时巨大，还会产生大量 CPU 与 GPU 之间的数据传输，几乎无法满足实时渲染需求。

3. Tile-Based Rendering

原版的 3DGS 借鉴了现代 GPU 的 Tile-Based Rendering 思想。将屏幕划分为固定大小的小块（Tile），例如：

```
+----+----+----+
| T0 | T1 | T2 |
+----+----+----+
| T3 | T4 | T5 |
+----+----+----+
| T6 | T7 | T8 |
+----+----+----+
```

通常每个 Tile 为 16 × 16 像素。随后，每个 Gaussian 根据自己的二维包围盒，被分配到它覆盖到的所有 Tile。例如：

```
Gaussian A
        ┌────────┐
        │        │
+----+----+----+
| T0 | T1 | T2 |
+----+----+----+
| T3 | T4 | T5 |
+----+----+----+
```

Gaussian A 同时覆盖：T1、T2、T4 和 T5。因此只需要加入这些 Tile 的渲染列表即可，而无需参与整个屏幕的排序。

4. 局部排序

完成 Tile 划分之后，每一个 Tile 都维护自己的 Gaussian 列表：

```
Tile 0

Gaussian 15
Gaussian 28
Gaussian 91
Gaussian 302
...
```

随后，仅在当前 Tile 内按照深度排序：

```
Far
↓

Gaussian302

Gaussian91

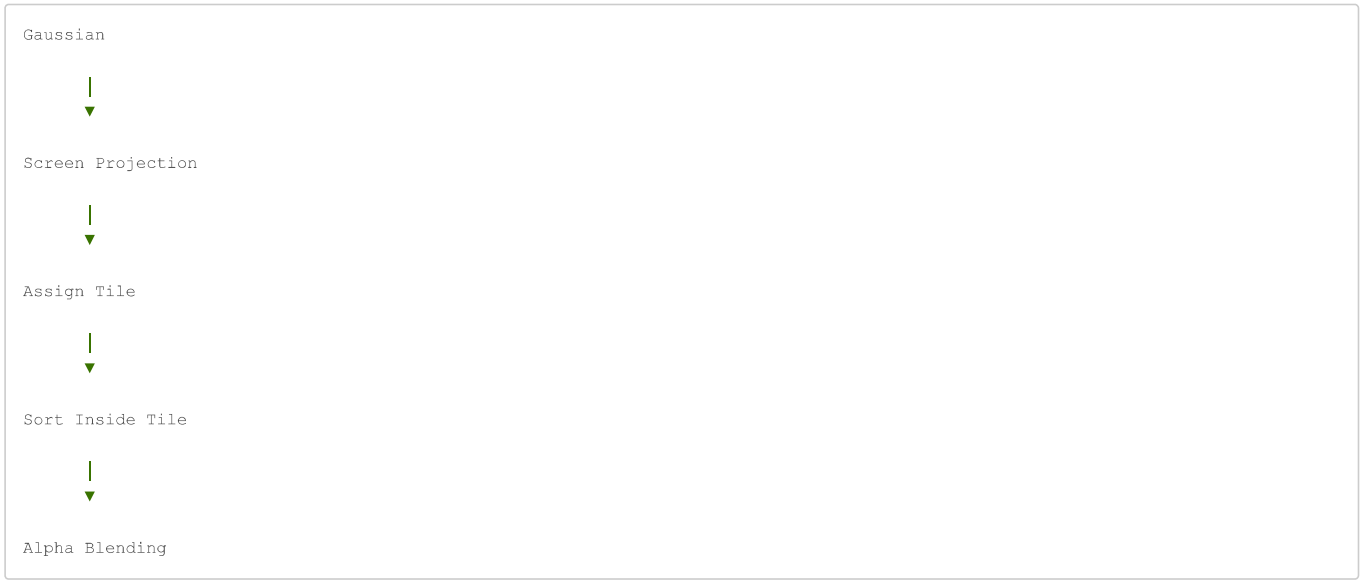
Gaussian28

Gaussian15

Near
```

最后由 GPU 对 Tile 内所有 Gaussian 完成透明混合。由于每个 Tile 覆盖的 Gaussian 数量通常只有几十到几百个，因此排序成本相比全局排序大幅降低。

整个渲染流程可以表示为：



可以看到，真正参与排序的不再是整个场景，而只是每一个 Tile 内部的 Gaussian。

5. 图形实现

以上是原版 3DGS 使用 CUDA 进行高效排序的实现思路。那么，如果想在现代图形引擎中实现同样的效果，该怎么做呢？答案是可能并不太容易实现，至少没有形成工程上的最优解。虽然现代 GPU 的算力非常强大，但在 GPU 中进行高效的全局或局部排序，本身就不是一件容易的事情。传统的图形渲染管线并没有提供现成的“排序”指令。

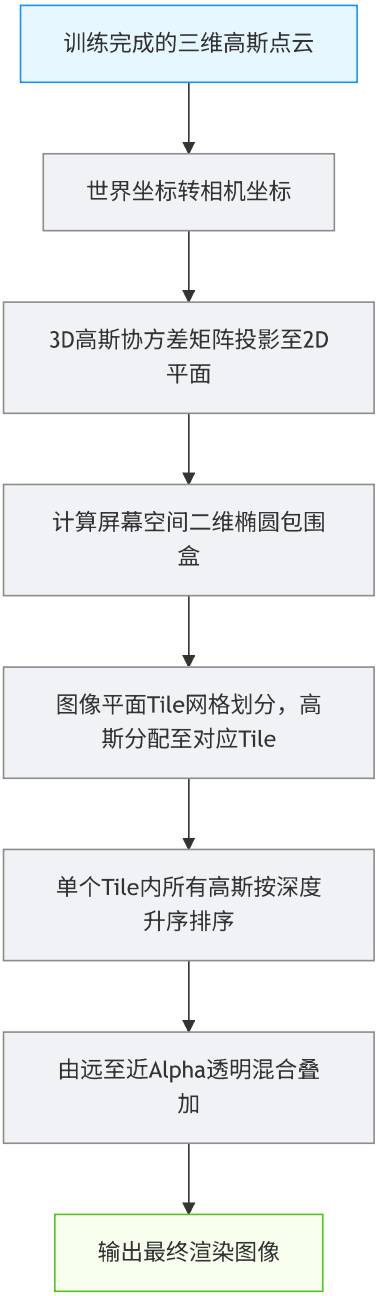
一种可行的工程思路是**利用计算着色器（Compute Shader）来复用上述的 Tile-Based 排序逻辑**。具体流程可能如下：

1. **投影与分配**：在 Compute Shader 中，首先将所有 3D 高斯投影到 2D 屏幕空间，并计算它们覆盖了哪些 Tile，生成 `(Tile ID, Depth, Gaussian ID)` 的列表。
2. **GPU 排序**：在 Compute Shader 中实现或调用一个高效的并行排序算法（如 GPU 版本的 Radix Sort 或 Bitonic Sort），对这个列表进行排序。
3. **光栅化与混合**：排序完成后，再将排序好的高斯列表传递给后续的渲染阶段（可能是另一个 Compute Shader 或传统的片元着色器），按照 Tile 顺序进行透明混合。

因此，尽管 3DGS 的渲染质量极高，但其在通用图形引擎中的集成门槛，很大程度上就卡在了这个“GPU 排序”的环节上。

九、总结

回顾整篇文章，我们已经完整分析了 3D Gaussian Splatting 的整个可视化流程。从最初的三维高斯表示，到最终生成屏幕上的每一个像素，其渲染过程可以概括为以下几个阶段：



可以看到，3DGS 并不是一种完全脱离传统图形学的新技术。恰恰相反，它充分继承了现代 GPU 图形渲染管线中的许多经典思想，例如：

- 世界空间到屏幕空间的坐标变换；
- 基于屏幕空间的 Rasterization；
- Tile-Based Rendering；
- Alpha Blending；
- Back-To-Front 透明混合；
- GPU 并行排序。

真正发生变化的，是**场景的表示方式（Scene Representation）**。传统图形渲染以三角形（Triangle Mesh）作为基本图元，通过顶点、索引和纹理来描述场景；而 3DGS 则使用大量带有位置、协方差、颜色和透明度属性的三维高斯来近似真实世界。渲染过程中，不再进行三角形光栅化，而是直接将每一个高斯投影为屏幕空间中的二维椭圆，并完成透明混合。

换句话说，**3DGS 改变的是"绘制什么"，而不是"如何绘制"**。从这一角度来看，3DGS 更像是在传统图形管线之上，引入了一种新的几何表达形式，而不是重新设计了一套全新的渲染体系。

当然，3DGS 也并非完美无缺。由于其采用大量半透明高斯进行颜色累积，因此必须依赖深度排序才能得到正确的渲染结果。这使得它很难直接复用传统图形管线中成熟的深度测试、延迟渲染（Deferred Rendering）以及各种基于 G-Buffer 的后处理技术。同时，在与三角形场景混合渲染时，也需要额外处理透明混合、渲染顺序以及深度一致性等问题。

此外，由于高斯本身是一种连续概率分布，而非具有明确拓扑结构的几何体，因此在碰撞检测、物理模拟、布尔运算、精确测量以及编辑等方面，也无法像传统 Mesh 那样直接进行处理。这也是近年来大量研究工作尝试将 3DGS 与 Mesh、NeRF、3D Tiles 等数据结构结合的重要原因。

尽管如此，3DGS 仍然代表了近年来三维视觉领域最重要的进展之一。它首次在重建质量、训练效率和实时渲染性能之间取得了较好的平衡，使基于图像的真实场景重建真正具备了实时交互能力，也推动了数字孪生、AR/VR、自动驾驶、机器人感知等多个方向的发展。

随着后续研究不断推进，围绕 3DGS 已经衍生出动态场景（4DGS）、层次化高斯（Hierarchical Gaussian）、压缩存储、LOD、自适应渲染、可编辑 Gaussian 以及与传统图形管线深度融合等大量工作。可以预见，在未来相当长的一段时间内，3DGS 仍将是三维重建与实时渲染领域的重要研究方向。

分类: 计算机图形学

标签: 计算机图形学 , 三维重建 , 实时渲染 , 3D Gaussian Splatting , 球谐函数

免责声明：本内容来自平台创作者，博客园系信息发布平台，仅提供信息存储空间服务。

好文要顶 关注我 收藏该文 微信分享



charlee44
粉丝 - 221 关注 - 10
会员号：2280
+加关注

4 0

升级成为会员

posted @ 2026-07-16 21:48 charlee44 阅读(440) 评论(0) 收藏 举报

博客园 © 2004-2026